

AuditPledge: Provenance and Priced Risk for the MCP Tools a Compound-Facing Agent Calls

Dipankar Sarkar
doom2quake

August 2026

Abstract

An AI agent that manages a Compound position is only as trustworthy as the weakest tool in its toolbelt. Agents increasingly acquire those tools over the Model Context Protocol (MCP), and one unsigned, remote-code-capable, over-scoped tool can read a signing key, rewrite a transaction before it is broadcast, or move collateral. The operator wiring such an agent has no standard way to answer which of the tools it can call are dangerous, how dangerous, and in money. We present AuditPledge, a supply-chain auditor that opens a real MCP session against an agent’s tool servers, verifies Ed25519 provenance for every tool against a committed public keyring, scores the risk weighted for what a Compound position exposes (state reads, transaction building, collateral movement), and prices the residual risk into an expected loss and a suggested premium. A counterfactual pass reruns the identical scorer over a mutated manifest, so the dollar saving from a fix is measured, not asserted. Everything ambiguous fails closed: unknown servers, identity mismatches, forged or missing signatures never read as clean. The audit, the cryptography, the scoring and pricing, and the UI numbers are real; the demo tool catalogue, the offline advisory snapshot, and the demo publisher keys are synthetic and labelled as such. The deterministic core is verified by 75 Python tests, needs no network, key, or cloud credentials, and all Web3 interaction is testnet and seeded only. This is a candidate entry to the Compound Grants Program, Security Tooling domain (Section 5).

1 Problem

DeFi is being wired up to AI agents: position managers, liquidation bots, rebalancers, and treasury assistants increasingly run as agents that call external tools over MCP [1]. MCP has become the default way an agent acquires tools, ahead of any security convention for it. A tool manifest is attacker-controlled data. A tool that merely *claims* a publisher and *claims* a signature proves nothing, yet an operator today reads a list of tool names, trusts the publisher on faith, and finds out something was wrong only after funds move.

Software supply-chain attacks — typosquatted packages, compromised maintainers, lookalike publishers — are already a dominant way real systems get breached [5]. The MCP tool layer imports every one of those failure modes and adds a live wire to a wallet. The failure sharpens under a Compound-facing agent, because the tools that matter are exactly the ones that sit in front of the money: readers that pull Comet [2] market and account state, builders that assemble and sign COMP-related transactions, and movers that supply or withdraw collateral. The concrete gap is the absence of three things at once: a provenance check on the tool supply chain, a severity ranking weighted for Compound exposure, and a dollar figure a risk owner or an insurer could act on.

2 Why Compound

Compound is a mature lending protocol with real value at stake and an active, security-minded governance process [3] whose Grants Program names Security Tooling as a funded domain. A security tool is not a one-shot deliverable; it is an artifact the ecosystem keeps using, which is the shape milestone funding supports. AuditPledge is not a chain-agnostic tool with a Compound label: its risk model weights capability against what a Compound position actually exposes, and its reference keyring is seeded from publishers the Compound community can vet. A provenance-and-pricing standard proven against Compound-facing agents is a public good the whole ecosystem inherits, and the moment to set the norm — that a tool a Compound agent calls carries verifiable provenance — is before an incident sets it the hard way.

3 Design

AuditPledge is a pipeline of pure functions over an MCP tool manifest and a committed keyring, so a reviewer can trace every number. The stages are: inventory, provenance verification, risk scoring, underwriter pricing, and counterfactual repricing.

3.1 The evidence path: a real MCP session

With `--source mcp`, AuditPledge opens an MCP stdio session, completes the `initialize` handshake, checks the server identity it was handed against the server it asked for, and reads `tools/list` over JSON-RPC. Unknown servers, identity mismatches, and unreachable servers all **fail closed**: the audit stops rather than reporting a clean manifest. A bundled synthetic Compound tool catalogue is read the same way over the same session, and is labelled `synthetic: true` everywhere it surfaces.

3.2 Provenance: Ed25519 over canonical bytes

Each tool carries a detached Ed25519 [4] signature over the canonical bytes of a fixed field set:

signed = (name, version, publisher, sorted scopes, sorted ops).

The verifier recovers nothing from self-assertion. It normalises the publisher and matches it *exactly* against the keyring — `attacker.compound-finance` is not `compound-finance`, because suffix matching is never used — and then verifies the signature over the canonical bytes with that publisher’s committed public key. Only a signature that verifies counts as signed; a present-but-invalid signature is recorded as *forged*. Tampering with any covered field (adding an op, widening a scope, bumping a version) invalidates the signature. Every ambiguous path (no signature, unknown publisher, non-hex signature, crypto library unavailable) returns `verified = false`, so the auditor treats the tool as unsigned rather than guessing.

3.3 Risk: weighted for Compound exposure

Each tool’s score is a sum of named weights over its declared capabilities and provenance. RCE-class ops (`exec`, `run_command`, `write_file`, and critically `sign_tx`, since on a Compound-facing agent a compromised signer moves real value) carry the heaviest weight; network ops, unsigned provenance, an untrusted publisher, and over-broad scopes (four or more, or a wildcard) each add their own. A distinct **Compound position exposure** weight is added when a tool can build or

sign a transaction or move collateral, because that is what sits directly in front of the money in a Comet position. Advisory hits from a source that must name its ID, origin, and retrieval time add further. Score bands map to `low/medium/high/critical`.

3.4 Pricing and counterfactuals

An underwriter pass converts the assessment into money. Per tool, a compromise probability p is read from its severity band, floored at a higher value when a tool is both unsigned and RCE-capable (the dominant real-world failure), and the expected loss is $p \times$ a configurable breach-loss assumption. Summed and multiplied by a risk load, that is a suggested annual premium. The value of a fix is then *measured*, not asserted: a counterfactual pass reruns the *identical* `assess` $\rightarrow$ `price` pair over a mutated manifest — the worst tool dropped, or re-published by a trusted publisher with its RCE/signing ops and wildcard scopes removed and *genuinely re-signed* with that publisher’s key — and reports the premium delta. Because the mutated manifest passes the same Ed25519 verification as any other, the repricing is a real rerun of the scorer, not a hand-edited number.

3.5 The agent graph

An optional layer turns the audit into a ranked remediation plan. It is an ADK [6] `SequentialAgent` that runs an Auditor sub-agent, then a Risk Lead sub-agent, in that fixed order; a run that did not write both output keys into session state is discarded rather than passed off as a plan. The order is enforced by the workflow agent, not requested in a prompt. The deterministic audit is the whole security and pricing claim and runs without any cloud credentials; the LLM layer only adds narrative on top of numbers the code already produced.

4 Implementation and evaluation

Provenance verification uses a standard Ed25519 implementation [4]; everything else — inventory, scoring, pricing, counterfactuals — is dependency-light Python that is a pure function of its inputs. The static UI (`auditpledge/ui/index.html`) carries no hand-written numbers: every figure is captured from one real audit run and pasted between snapshot markers, and a snapshot test reruns the audit and fails if any value drifts.

We evaluate by test with `pytest`:

- **75 Python tests** (`PYTHONPATH=.` `pytest -q`, Python 3.10+, the audit store forced in-memory). They pin each defence: the Ed25519 verification and its tamper cases, exact-publisher matching, the unsigned-RCE pricing floor, the counterfactual repricing, the advisory-ID validation (a placeholder such as `GHSA-xxxx-xxxx-xxxx` is refused rather than printed as a known vuln), the fail-closed MCP paths against a real stdio subprocess (`handshake`, `tools/list`, identity mismatch, missing evidence, unreachable server), the agent-graph ordering, and the UI snapshot. No network, key, or cloud credential is needed.

The sharpest case is the hostile manifest `evil-lookalike`: a tool that claims the `compound-finance` publisher with a bare-string “signature” and a lookalike `attacker.compound-finance` entry. The auditor recovers no trust from either: the first is recorded as forged, the second fails the exact-publisher match, and both are scored as unsigned. That pins the property that self-asserted provenance is not trust.

5 Limitations

This is milestone 1, and the honest scope is narrow (the full map is in `docs/HONESTY.md`).

- **No mainnet, no users, no revenue.** All Web3 interaction is testnet and seeded only; nothing is on Compound mainnet.
- **No third-party MCP server has been audited by us.** AuditPledge demonstrates both ends against its own synthetic catalogue. A real third-party tool does not yet publish provenance evidence, which is exactly what the milestone roadmap delivers; against such a tool the auditor fails closed rather than inventing a verdict.
- **The pricing inputs are illustrative priors, not an actuarial model.** The per-severity probabilities, the unsigned-RCE floor, the breach-loss figure, and the risk load are defensible, runtime-configurable starting points. What is demonstrated is the mechanism: a manifest becomes a reproducible number and a fix changes it by a measured amount.
- **No audit, partnership, or endorsement** from Compound or anyone else. This is a grant application.

6 Conclusion

AuditPledge turns “which of my Compound agent’s tools are dangerous, how dangerous, and in money” into a reproducible, provenance-checked, priced answer that fails closed on every ambiguous path. The counterfactual makes the value of a fix a measured saving rather than a claim, and the UI cannot show a number the auditor did not produce. The durable contribution is a provenance-and-pricing discipline for the MCP tool supply chain that Compound-facing agents depend on, established before an incident forces it.

References

- [1] Anthropic. *Model Context Protocol Specification*. <https://modelcontextprotocol.io/>, 2024–2026.
- [2] Compound Labs. *Compound III (Comet) Documentation*. <https://docs.compound.finance/>, 2023–2026.
- [3] Compound Governance Forum. <https://www.comp.xyz/>, 2026.
- [4] D. J. Bernstein et al. *High-speed high-security signatures (Ed25519)*. Journal of Cryptographic Engineering, 2012.
- [5] Open Source Vulnerabilities (OSV). *OSV.dev distributed vulnerability database*. <https://osv.dev/>, 2021–2026.
- [6] Google. *Agent Development Kit: Workflow Agents (SequentialAgent)*. <https://google.github.io/adk-docs/>, 2025–2026.